FRONTEND ASSIGNMENT - 3

Subject: Frontend Development

Submitted By: Vidhi Attri

University Roll No.:22515000159(42)

Course: B. Tech - Computer Science and Engineering

Section: NA (L2)

College: GLA University

SUBMITTED TO: MR GAUTAM MUKHERJEE

Q1. What is the difference between Frontend and Backend JavaScript?

Frontend JavaScript: It runs in the user's web browser. It makes web pages interactive, validates forms, handles button clicks, and updates webpage content.

Backend JavaScript: It runs on the server using environments such as Node.js. It handles server requests, connects to databases, manages authentication, and processes data.

Examples: Frontend: displaying an error when a password is incorrect. Backend: checking login details in a database.

Frontend JavaScript	Backend JavaScript
Runs in the browser.	Runs on the server.
Manages the user interface.	Manages server-side operations.
Uses the DOM to modify webpage content.	Handles databases and server requests.
Example: JavaScript in HTML.	Example: JavaScript using Node.js.

Q2. What is DOM?

DOM stands for **Document Object Model**. It represents an HTML document as a tree of objects. JavaScript uses the DOM to access, modify, add, or remove HTML elements and change their styles.

Example

```
<p id="demo">Hello</p>

<script>
document.getElementById("dem
o").innerHTML
= "Welcome"; </script>
```

Explanation: Initially, the paragraph displays “Hello”. JavaScript selects it using its ID and changes the text to “Welcome”.

Q3. What is an Event in JavaScript?

An event is an action or occurrence in a web browser that JavaScript can detect. Events may be triggered by the user or by the browser.

Examples: `click` (element clicked), `mouseover` (mouse moves over an element), `keydown` (key pressed), `submit` (form submitted), and `load` (resource finishes loading).

Example

```
<button onclick="showMessage()">Click Me</button>

<script> function showMessage()
{
alert("Button clicked!");
}
</script>
```

Explanation: When the button is clicked, `showMessage()` runs and displays an alert.

Q4. What is the Event Loop in JavaScript?

The event loop allows JavaScript to handle asynchronous operations while executing most JavaScript code on a single main thread. The call stack executes functions, while browser or runtime APIs handle operations such as timers. Ready callbacks are scheduled and run when the stack is available.

Example

```
console.log("Start");

setTimeout(function() {
console.log("Timer"); }, 0);

console.log("End");
```

Output: Start, End, Timer

Explanation: “Start” and “End” are printed first. The timer callback runs later, after the current synchronous code finishes.

Q5. What is Hoisting in JavaScript?

Hoisting is JavaScript's behavior of processing declarations before executing code in their scope. Different declarations behave differently.

• `var` declarations are hoisted and initialized with `undefined`.

• `let` and `const` are hoisted but cannot be accessed before initialization.

• Function declarations can generally be called before their position in the code.

Example

```
console.log(a);
var a = 10;
```

Output: undefined

Explanation: The declaration of `a` is processed before execution, but its assignment of 10 happens later.

Q6. What is Event Bubbling?

Event bubbling is a process in which an event triggered on an inner HTML element propagates upward through its parent elements.

Example

```
<div onclick="alert('Div
clicked')"> <button
onclick="alert('Button
clicked')">
Click Me
</button>
</div>
```

Explanation: Clicking the button shows the button's alert first, followed by the div's alert. The click event bubbles from the button to its parent. Bubbling can be stopped using `event.stopPropagation()`.

Q7. Difference between Named, Anonymous and Arrow Functions

These are three ways to define functions in JavaScript.

1. Named Function

```
function add(a, b) { return a + b; } console.log(add(2, 3));
```

Output: 5. A named function has a specific name and can be called using that name.

2. Anonymous Function

```
const add = function(a, b) { return a + b; };

console.log(add(2, 3));
```

Output: 5. An anonymous function has no name of its own and is often assigned to a variable or passed as a callback.

3. Arrow Function

```
const add = (a, b) => a + b;

console.log(add(2, 3));
```

Output: 5. An arrow function uses the => symbol and offers shorter syntax.

Named Function	Anonymous Function	Arrow Function
Has a function name.	Has no name of its own.	Uses the => symbol.
Uses the function keyword.	Usually uses the function keyword.	Uses shorter syntax.
Has its own this, based on how it is called.	Has its own this, based on how it is called.	Inherits this from surrounding scope.

Q8. Explain for-of and for-in Loops.

The `for-of` and `for-in` loops iterate over data in different ways.

1. for-of Loop

The `for-of` loop accesses values of iterable objects such as arrays and strings.

```
let numbers = [10, 20, 30];

for (let num of numbers) { console.log(num); }
```

Output: 10, 20, 30. The loop accesses each array value one by one.

2. for-in Loop

The `for-in` loop iterates over enumerable property keys of an object. For arrays, it generally gives indexes as strings.

```
let student = { name:
"Pushpendra", age:
20, course: "B.Tech"
};

for (let key in student) {
console.log(key, student[key]);
}
```

Output: name Pushpendra; age 20; course B.Tech.

Difference: Use `for-of` for values from iterables such as arrays. Use `for-in` to iterate over an object's property keys.

Q9. Explain Any Five Array Methods in JavaScript.

JavaScript provides built-in array methods to perform operations on arrays. Five important methods are:

1. push()

Adds one or more elements to the end of an array.

```
let arr = [10, 20, 30];
arr.push(40);
console.log(arr);
```

Output: [10, 20, 30, 40]

2. pop()

Removes and returns the last element of an array.

```
let arr = [10, 20, 30];
arr.pop(); console.log(arr);
```

Output: [10, 20]

3. shift()

Removes and returns the first element of an array.

```
let arr = [10, 20, 30];
arr.shift();
console.log(arr);
```

Output: [20, 30]

4. unshift()

Adds one or more elements to the beginning of an array.

```
let arr = [20, 30];
arr.unshift(10);
console.log(arr);
```

Output: [10, 20, 30]

5. map()

Creates a new array by applying a function to every element of the original array.

```
let numbers = [1, 2, 3]; let result =
numbers.map(num => num * 2); console.log(result);
```

Output: [2, 4, 6]. Each element is multiplied by 2 and stored in a new array. The original array remains unchanged.

Q10. What is a Callback Function in JavaScript?

A callback function is a function passed as an argument to another function and called by that function when required. Callbacks are commonly used for event handling, asynchronous operations, and tasks performed after another operation.

Example

```
function greet(name, callback) {
console.log("Hello " + name); callback();
}

function message() {
console.log("Welcome to JavaScript");
}

greet("Pushpendra", message);
```

Output: Hello Pushpendra; Welcome to JavaScript.

Explanation: The `greet()` function accepts a name and a callback. It prints the greeting and then calls `message()` through `callback()`.

Conclusion: Callback functions help execute one function after or during another function's work. They are important in JavaScript, especially for event handling and asynchronous tasks.